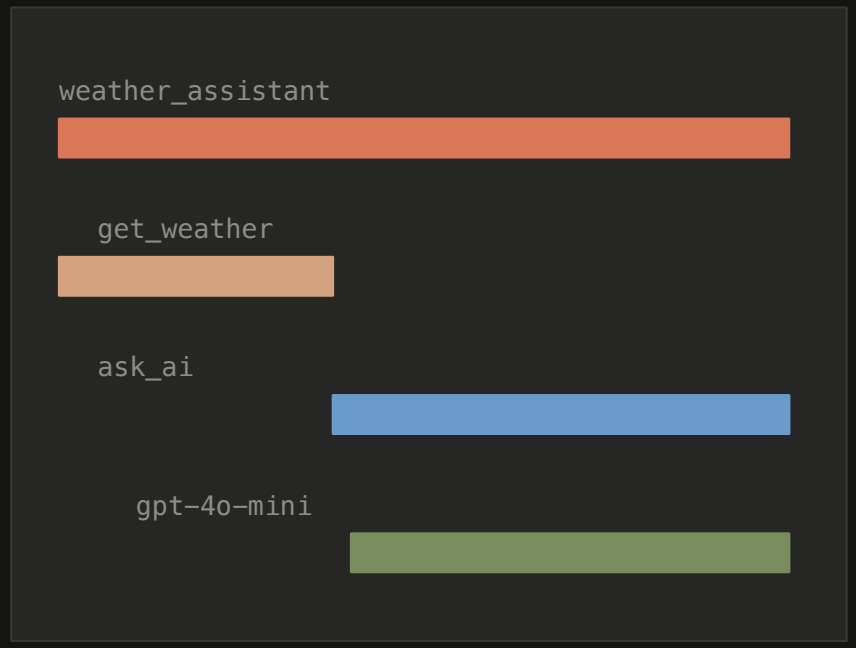
Data Sense · AI Discussions

AI *Observability*

See what your AI app is doing, step by step.



Foundations · Tuesday, October 20, 2026 · 7:00 PM PT

Your AI assistant gave a wrong answer, and *nothing told you why*

Customer

Can I get a refund after 60 days?

Assistant

Yes, refunds are available anytime.

The real policy is 30 days. The server dashboard showed no errors.

To fix it you need answers to:

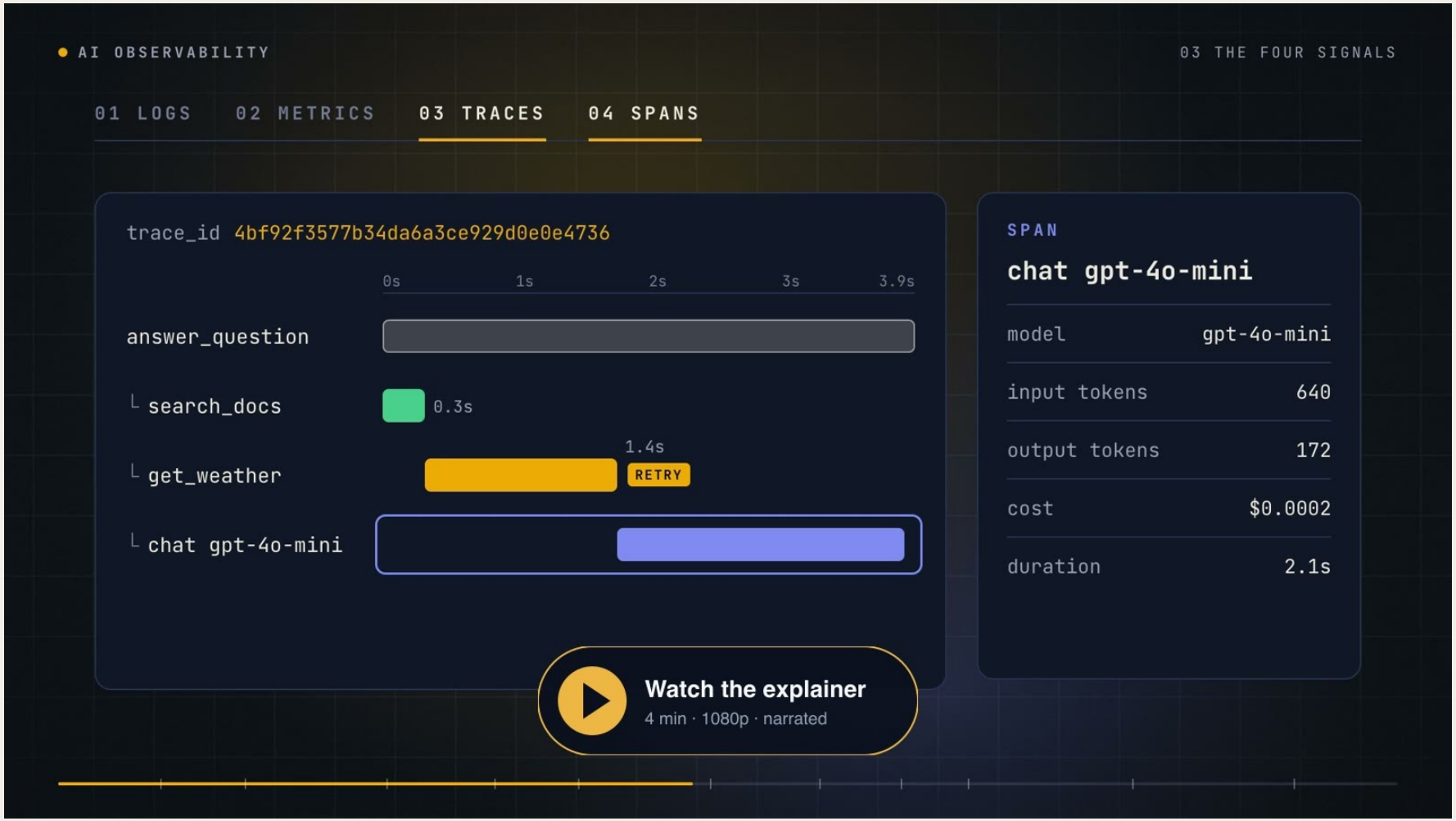
- Which question was sent to the AI model?
- What did each step return?
- How long did each step take?
- What did it cost?

Agenda

Tonight: *the basics first*, then three tools on the same tiny example

1	**The basics**	Logs, metrics, traces and spans in plain words	Notebook 01
2	**Langfuse**	Send your first trace to a real dashboard	Notebook 02
3	**LangSmith**	The same example in a second tool	Notebook 03
4	**Other tools**	OpenTelemetry, Datadog, Dynatrace, and how to choose	Notebook 04

Four-minute overview



Part 1

The basics

What observability means, and the four kinds of information an app can leave behind.

1

Monitoring tells you something is wrong; *observability tells you why*

Monitoring

Is it broken?

- Questions decided in advance
- Dashboards and alerts
- Catches problems you expected

Observability

Why is it broken?

- Questions you ask afterwards
- Answered from data the app leaves behind
- Explains problems you did not expect

AI apps break the old rules, so *a green dashboard is not enough*

	Traditional app	AI app
Same input	Same output every time	A different answer each time
When it fails	An error message, a crash	A confident wrong answer, no error
Cost	Fixed server cost	Every request costs money (tokens)
Where the logic lives	In the code	In prompts, the model and its tools
Speed	Milliseconds	Seconds, and it varies a lot

You already use observability every time you *track a parcel*

Software word	Parcel example	What it means
Log	“Door locked, will retry tomorrow”	A short message about one moment
Metric	“12,400 parcels delivered today”	A number that summarizes many things
Trace	The full tracking history of your parcel	Everything that happened to one request
Span	“Left warehouse, took 3 hours”	One step inside a trace, with its duration

Our example is a *two-step weather assistant*, used in every notebook

```
def get_weather(city):
    time.sleep(0.5)            # pretend: slow website
    return FAKE_WEATHER[city]

def ask_ai(question):
    return ...call gpt-4o-mini...

def weather_assistant(city):
    weather = get_weather(city)
    return ask_ai(f"It is {weather}. Umbrella?")
```

Step 1

Look up the weather for a city

Step 2

Ask an AI model: should I carry an umbrella?

Plain Python. The rest of tonight only adds a few lines to it.

Logs record what happened, but get hard to follow with many users

```
17:54:28  INFO  request started for London
17:54:29  INFO  weather is stormy, 11°C
17:54:30  INFO  AI answered: Yes, you should carry an
umbrella.
```

Output from notebook 01: three log lines for one request.

Good at

Details: error messages, values, what the code decided.

Weak at

With a thousand users at once, lines from different people are mixed together.

Metrics summarize many requests, but not which step was slow

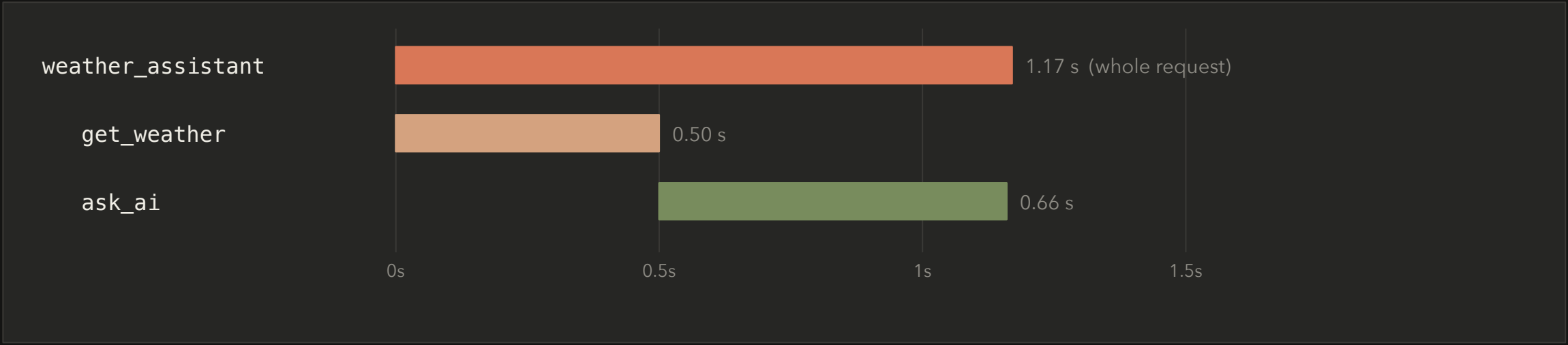
4	1.20 s	1.29 s
requests served	average time	slowest request

On a real system these numbers live on a dashboard, with an alert when the average goes above a limit.

But “the average is 1.2 seconds” does not say whether the weather lookup or the AI model was slow.

Numbers from notebook 01, four cities.

A trace shows one request step by step, and where time went



Trace The whole story of one request (the top bar).

Span One step, with a start and an end (each bar). Steps sit inside the request.

Real trace of weather_assistant("Tokyo") from notebook 01. Time runs left to right.

Creating spans takes *one line per step* with OpenTelemetry

```
def weather_assistant(city):
    with tracer.start_as_current_span("weather_assistant"):
        with tracer.start_as_current_span("get_weather"):
            weather = get_weather(city)
        with tracer.start_as_current_span("ask_ai"):
            return ask_ai(...)
```

`with ... :`

Everything inside the block is timed and saved as one span.

A block inside a block

becomes a child step, so you get the tree on the previous slide.

OpenTelemetry is a free, standard library. Setting it up is four lines, once per app (notebook 01).

Four pieces move a span from your code to a tool

Piece	What it does	Parcel comparison	Examples
Tracer provider	Holds the settings for your whole app	The courier company	TracerProvider()
Tracer	Creates spans in your code	The clerk writing tracking entries	provider.get_tracer("weather-assistant")
Span processor	Decides when finished spans are handed on: one at a time, or in batches	Mailroom: send each parcel now, or wait and fill a truck	SimpleSpanProcessor (now), BatchSpanProcessor (in batches, for real apps)
Exporter	Sends spans to a destination	The delivery truck	ConsoleSpanExporter (print), InMemorySpanExporter (keep), OTLPSpanExporter (to Langfuse, LangSmith, Datadog...)
Collector (optional)	A separate program that receives spans from many apps, cleans them and forwards them	A regional sorting hub	OpenTelemetry Collector, Grafana Alloy, Datadog Agent

Notebook 01 uses Simple + InMemory so you can see the spans. Notebooks 02 and 04 use batching and OTLP to send them to Langfuse.

For AI steps, a span also records the *model, tokens and cost*

What a span stores	Example: the AI call in our assistant
Name	OpenAI-generation
Start and end time	0.50 s to 1.03 s
Inside of	ask_ai, inside weather_assistant
Model	gpt-4o-mini
Tokens in / out	31 / 8
Cost	$0.000009
Input and output	The question sent, the answer received

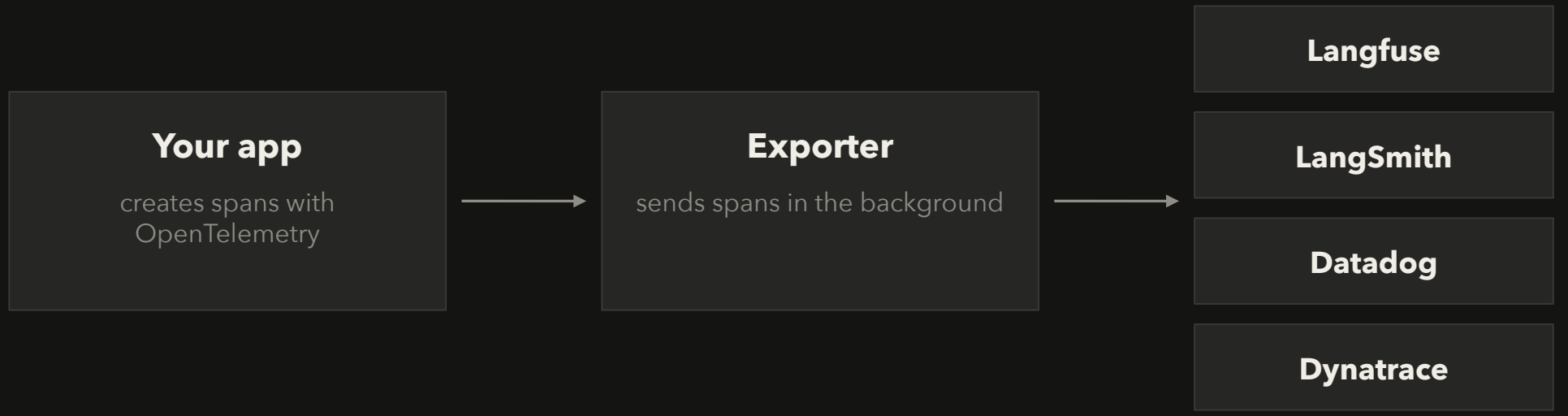
Values from a real trace in the local Langfuse (notebook 02).

Tokens

Pieces of words. AI providers charge per token, so tokens are how you see cost.

You do not record these by hand. Langfuse and LangSmith fill them in for you.

OpenTelemetry is the *common language* all tools understand



Each tool stores the spans and shows them on a web page.

Switching tools means changing where the exporter sends data. Your functions stay the same.

Five things are worth watching in every AI app

1
Speed

How long requests take, especially the slowest ones

2
Errors

Timeouts, failed tools, rate limits

3
Tokens

How much text goes in and out of the model

4
Cost

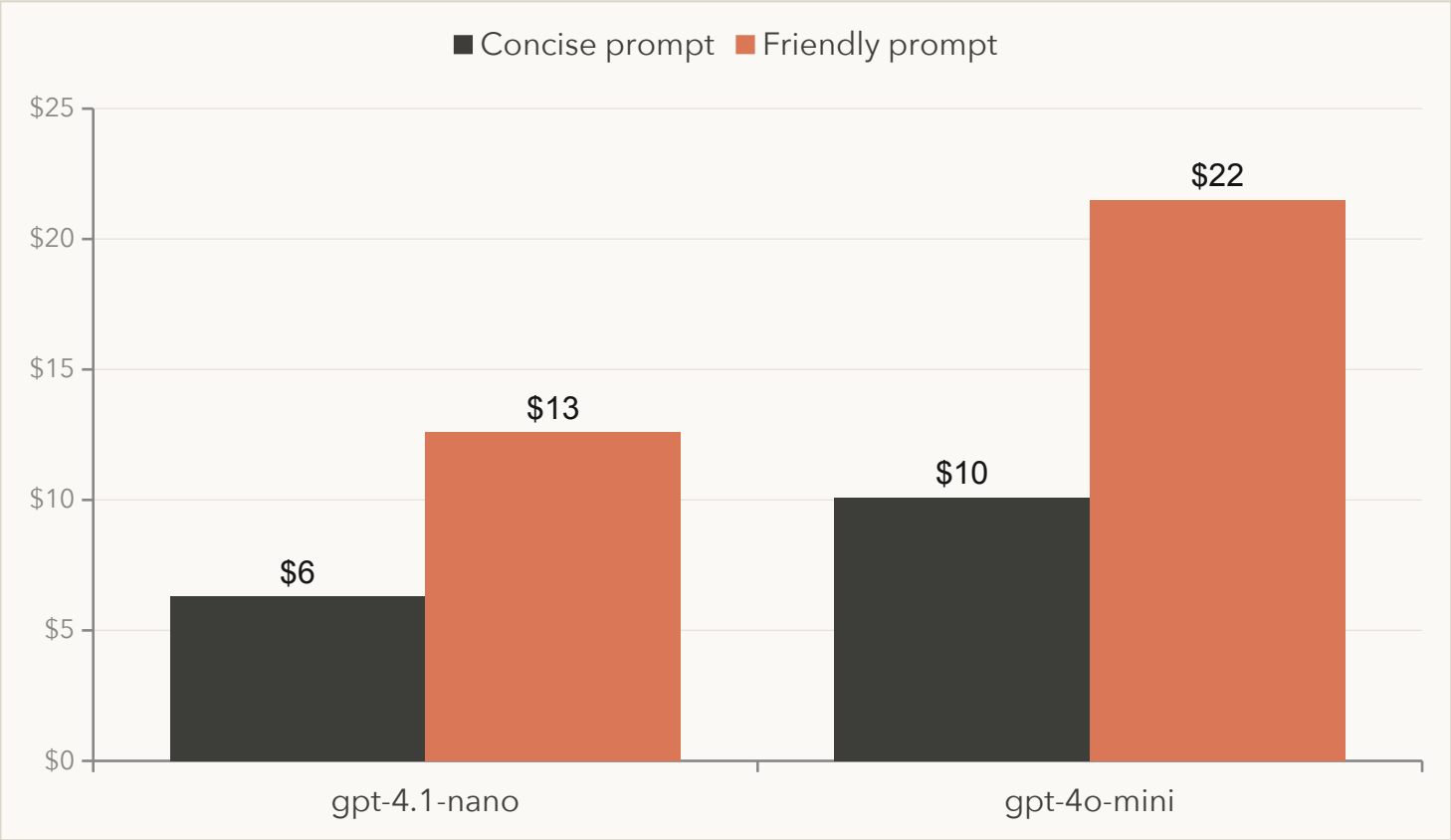
Money per request, per user, per day

5
Quality

Thumbs up or down, or another AI grading answers

Also record who asked (user) and which prompt version you used, so you can compare later.

A friendlier prompt made answers longer and *doubled the monthly bill*



31 tokens

average answer, concise prompt

87 tokens

average answer, friendly prompt

Longer answers mean more tokens, and more tokens mean more money. You only see this if you measure it.

Projected cost at 10,000 requests per day, USD per month. Measured on 48 real requests (notebooks/advanced/05).

Part 2

Langfuse

Send the same assistant to a real dashboard, running on this laptop.

2

Langfuse is a *free, open source* dashboard for AI traces

```
make up
open http://localhost:3000
```

Login: admin@observability.lab / observability123

What starts in Docker

Part	Job
Web app	The dashboard; receives traces
Worker	Processes traces, works out cost
Databases	Postgres, ClickHouse, Redis and MinIO store everything

In short: a website and a database on your own machine. Nothing leaves your laptop except the AI call itself.

Setting up Langfuse on your laptop takes *five steps*

1	**Install Docker Desktop** From docker.com, install, then open it once	`docker --version`
2	**Download Langfuse** The official setup files from GitHub	`git clone https://github.com/langfuse/langfuse.git`
3	**Start it** First start takes 2 to 3 minutes	`cd langfuse && docker compose up -d`
4	**Create the admin account** Click Sign up. No default admin: you pick the password	`http://localhost:3000`
5	**Create a project and API keys** Settings, API Keys. Copy both keys into .env	`LANGFUSE_PUBLIC_KEY=pk-lf-...`

Shortcut used tonight: the repo's make up starts the same stack and creates the admin login and keys automatically.

Adding Langfuse takes *two changes* to the same function

```python
from langfuse import observe
from langfuse.openai import OpenAI   # change 1

@observe()                           # change 2
def get_weather(city): ...

@observe()
def ask_ai(question): ...

@observe()
def weather_assistant(city): ...
```

Change 1

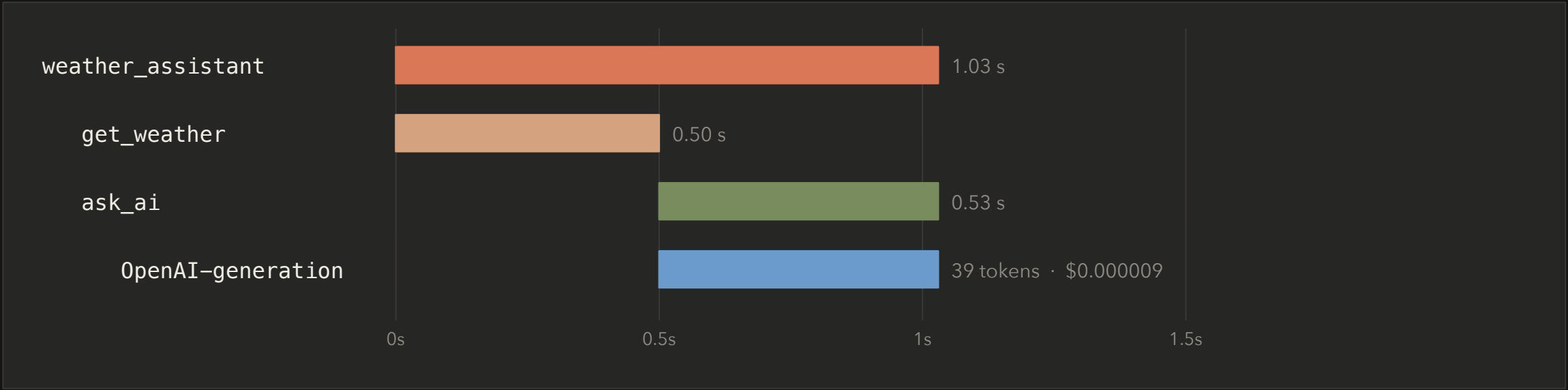
Import OpenAI from Langfuse. Every AI call is recorded: question, answer, model, tokens, cost.

Change 2

@observe() turns each function into a span, like the with blocks before.

Then call langfuse.flush() to send.

Langfuse adds the AI call details *we never wrote*



In the Langfuse page: click Tracing, open the newest trace, then click each row. The last row shows the exact question sent, the reply, the model, tokens and cost.

Real trace fetched from the local Langfuse (notebook 02).

Part 3

LangSmith

The same assistant in a second tool. Notice how little changes.

3

LangSmith needs the same two changes, with *different names*

Purpose	Langfuse	LangSmith
Record every AI call	from langfuse.openai import OpenAI	client = wrap_openai(OpenAI())
Turn a function into a span	@observe()	@traceable
Settings in .env	LANGFUSE_PUBLIC_KEY, ..._SECRET_KEY	LANGSMITH_API_KEY, LANGSMITH_TRACING=true
Where it runs	Your laptop or their cloud	Their cloud (smith.langchain.com)

Get a free key at smith.langchain.com, add it to .env, and run notebook 03.

Each tool uses its own words for *the same few ideas*

The idea	OpenTelemetry	Langfuse	LangSmith
One request, start to finish	Trace	Trace	Trace
One step	Span	Observation	Run
An AI model call	Span with AI details	Generation	LLM run
A folder of traces	Service	Project	Project
A conversation		Session	Thread

When a tool says observation, run or generation, read it as “a span”.

Part 4

Other tools

Langfuse and LangSmith are not the only options.

4

Tools come in *two groups*: built for AI apps, or general monitoring

Built for AI apps

Understand prompts, tokens and cost out of the box



Langfuse

Open source, self-host



LangSmith

Hosted, from the LangChain team



Arize Phoenix

Open source, lightweight

General monitoring platforms

Put AI traces next to servers and databases



Datadog

LLM Observability add-on



Dynatrace

AI features on full monitoring



Grafana

Open source dashboards

Switching tools means changing *an address, not code*

Tool	Where to send spans (endpoint)	Key (header)
Langfuse	http://localhost:3000/api/public/otel	Authorization: Basic <keys>
LangSmith	https://api.smith.langchain.com/otel	x-api-key: <key>
Dynatrace	https://<id>.live.dynatrace.com/api/v2/otlp	Authorization: Bearer <token>
Datadog	Your Datadog site's OTLP address	dd-api-key: <key>

Notebook 04 sends the notebook 01 assistant to Langfuse using only OpenTelemetry, with no Langfuse code at all.

Pick the tool that *fits your situation*

If...	Then try
You are learning or building a side project	Langfuse, on your laptop like tonight
Your app uses LangChain or LangGraph	LangSmith
Your data must not leave the company	Langfuse or Phoenix, self-hosted
Your company already pays for Datadog or Dynatrace	Their AI features, fed by OpenTelemetry

Four things to *remember*

1. **Observability** means your app leaves enough behind to explain what happened.
2. **Logs, metrics, traces, spans:** messages, numbers, the story of one request, and its steps.
3. **Langfuse and LangSmith** add tracing with two small changes and record AI details for you.
4. **OpenTelemetry** is the common language, so you can switch tools without rewriting code.

Questions?

Try it yourself

`github.com/fnusatvik07/ai-observability-lab`

`make setup` · `make up` · `make lab`

Next session

Grading answers automatically, managing prompt versions, alerts and dashboards.